

A server foundation / runtime

Component	Real-world analogy
Logger	Security camera + event log
Thread Pool	Fixed team of workers
Task Queue	Inbox of jobs
LRU Cache	Small counter near chef for frequently used items
CMake structure	Blueprint & construction rules

You didn't build a restaurant yet.

You built the **kitchen, staff system, storage, and management rules**.

TitanServer/

|—— include/ ← public interfaces

|—— src/ ← implementations

|—— CMakeLists.txt ← build rules

What problem does Logger solve?

In multi-threaded systems:

- `std::cout` is **not safe**
- log messages can interleave
- debugging becomes impossible

In C++, a **Singleton** is a design pattern that ensures a class has only **one instance** throughout the program and provides a **global access point** to that instance.



Key Characteristics of Singleton

- **Single Instance:** Only one object of the class is created during the program's lifetime.

- **Global Access:** Provides a static method (commonly getInstance()) to access the single instance.
- **Private Constructor:** Prevents direct instantiation using new.
- **Deleted Copy/Assignment:** Stops duplication of the instance.

```
#include <iostream>
using namespace std;

class Singleton {
private:
    static Singleton* instance; // static pointer to the single instance
    Singleton() { cout << "Singleton instance created\n"; }

    // Prevent copy and assignment
    Singleton(const Singleton&) = delete;
    Singleton& operator=(const Singleton&) = delete;

public:
    static Singleton* getInstance() {
        if (instance == nullptr) {
            instance = new Singleton();
        }
        return instance;
    }

    void showMessage() {
        cout << "Hello from Singleton!" << endl;
    }
};

// Initialize static member
Singleton* Singleton::instance = nullptr;

int main() {
    Singleton* s1 = Singleton::getInstance();
    Singleton* s2 = Singleton::getInstance();

    s1->showMessage();

    // Both pointers refer to the same instance
    cout << (s1 == s2 ? "Same instance" : "Different instances") << endl;
    return 0;
}
```

Output:

Singleton instance created

Hello from Singleton!

Same instance

logger.h (design)

class Logger {

public:

static Logger& instance();

void info(const std::string& message);

void error(const std::string& message);

Why a **singleton**?

Because:

- logs are a **shared global concern**
- multiple loggers = race conditions
- opening files multiple times = bad

Analogy:

You don't want 10 security cameras writing to 10 different logs.

Why static Logger& instance()?

```
static Logger logger;
```

```
return logger;
```

This gives you:

- **exactly one instance**
- thread-safe initialization (since C++11)
- no manual memory management
- no global variables

This is **modern C++ best practice**.

Mutex usage

```
std::lock_guard<std::mutex> lock(mutex_);
```

Why?

Because multiple threads may call info() simultaneously.

Analogy:

Only one person can write on a whiteboard at a time.

Why lock_guard instead of mutex.lock()?

- RAII (auto-unlock)
- exception-safe
- impossible to forget unlock()

What problem does a thread pool solve?

Bad approach

“Create one thread per request”

Why bad?

- Threads are expensive
- OS scheduling overhead
- Memory explosion
- Context switching kills performance

Good approach

Fixed number of worker threads

=> Reuse them forever

Concept	Where
RAII	mutex locking, thread joining
Thread safety	Logger, ThreadPool, LRU
Condition variables	worker loop
Move semantics	<code>std::move(task)</code>
Lambdas	task enqueue
Templates	LRU cache
Singleton	Logger

Q1: Why a thread pool instead of one thread per request?

Answer:

Threads are expensive OS resources. Creating one per request causes context switching overhead and memory pressure. A fixed-size thread pool amortizes thread creation cost and provides backpressure.

Analogy:

A restaurant hires a fixed number of chefs instead of hiring a new chef per customer.

Q2: Why use condition_variable?

Answer:

It allows threads to sleep efficiently until work arrives, avoiding busy waiting and reducing CPU usage. This is critical for scalable systems.

Q3: How do you ensure clean shutdown?

Answer:

A shared stop flag combined with condition variable notification ensures workers finish queued tasks and exit gracefully without race conditions.

Q4: Why is Logger a singleton?

Answer:

Logging is a cross-cutting concern that should be globally consistent. A singleton avoids multiple file handles and ensures serialized access.

Q5: Why use list + unordered_map for LRU?

Answer:

The map provides $O(1)$ lookup, and the list provides $O(1)$ reordering and eviction, achieving optimal performance.

Q6: What are the tradeoffs in your cache design?

Answer:

Single mutex limits concurrency but simplifies correctness. This can later be optimized with sharding if contention becomes a bottleneck.

(That last sentence is **chef's kiss**.)

Why HTTP parsing comes first

Because:

- sockets give you **raw bytes**
- the *real* intelligence of a server is understanding those bytes
- parsing is OS-agnostic and testable
- interviewers LOVE seeing this done manually

Express.js hides this.

You're about to understand what it hides.

What we are going to parse (HTTP/1.1 – minimum viable)

We need to extract:

- HTTP method (GET, POST)
- Path (/index.html)
- Version (HTTP/1.1)
- Headers (key → value)
- Body (later, for POST)

How does a web server understand an HTTP request?

HTTP parsing is like reading a postal envelope – you first read the address line, then scan the stamps and notes, and finally open the letter if present.

What an HTTP response actually is ?

Example raw response:

HTTP/1.1 200 OK

Content-Type: text/html

Content-Length: 13

Hello, world!

It has:

1. **Status line**
2. **Headers**
3. **Blank line**
4. **Body**

How does a server generate an HTTP response?

A response is constructed by writing a status line, followed by headers, a CRLF separator, and the body. Content-Length is computed from the body to allow the client to know when the response ends.

Analogy:

It's like sending a letter with a table of contents – the recipient knows how much to read

What is CRLF?

- **CRLF** stands for **Carriage Return + Line Feed**.

- It's a **newline sequence** used in text files and network protocols to mark the end of a line.
- Represented as:
 - `\r` → Carriage Return (CR, ASCII 13)
 - `\n` → Line Feed (LF, ASCII 10)
- Together: `\r\n`

Tie Request → Response => (Routing Layer)

How does a request become a response?

The server parses raw bytes into a structured request, routes the request based on method and path, generates a response object, and serializes it back into HTTP wire format.

Analogy:

It's like converting spoken language into an intent, deciding an action, and responding with a prepared script.

Sockets do not add intelligence.

They only:

- receive bytes
- send bytes

We already know what to do **with** the bytes.

So our job is simply:

`recv()` → string

parse → `HttpRequest`

handle → `HttpResponse`

`send()` → `response.toString()`

Winsock basics

Windows networking uses **Winsock**.

Required steps (always in this order):

1. **WSAStartup** → Initialize Winsock.
2. **socket** → Create a socket.
3. **bind** → Assign IP + port.
4. **listen** → Wait for connections.
5. **accept** → Accept a client.
6. **recv** → Receive data.

7. **send** → Send data.
8. **closesocket** → Close sockets.
9. **WSACleanup** → Cleanup Winsock.

1. **WSAStartup**

- Initializes the Winsock library.
- Must be called before any other Winsock functions.
- Loads the DLL and prepares resources.

2. **socket**

- Creates a socket (an endpoint for communication).
- You specify:
 - Address family (AF_INET for IPv4).
 - Socket type (SOCK_STREAM for TCP).
 - Protocol (IPPROTO_TCP).

3. **bind**

- Associates the socket with a local IP address and port.
- This tells the OS: “I want to listen on this port.”

4. **listen**

- Marks the socket as a **listening socket**.
- Allows it to accept incoming connection requests.
- You can specify a backlog (queue size).

5. **accept**

- Waits for a client to connect.
- When a client connects, accept() returns a new socket dedicated to that client.

6. **recv**

- Receives data from the connected client.
- Blocks until data arrives (unless socket is non-blocking).

7. **send =>** Sends data back to the client.

8. **closesocket**

- Closes the socket when communication is done.
- Frees resources associated with that socket.

9. **WSACleanup**

- Shuts down the Winsock library.
- Releases resources allocated by WSAStartup.

- Example:
- `cpp`
- `WSACleanup();`

HTTP — FUNDAMENTALS (MOST IMPORTANT)

Q1. What is HTTP, really?

Strong answer:

HTTP is a stateless, text-based application-layer protocol built on top of TCP. It defines a request-response model where clients send structured text requests and servers return structured text responses.

Key words interviewers listen for:

- stateless
- request-response
- application layer
- built on TCP

Analogy:

HTTP is like sending letters over a guaranteed delivery service (TCP). TCP ensures delivery; HTTP defines the letter format.

Q2. Why does HTTP need parsing?

Strong answer:

Because the server receives raw bytes from a socket. Parsing converts those bytes into a structured representation so the server can reason about intent, headers, and payload.

Follow-up depth:

- sockets give you bytes, not meaning
- parsing gives meaning

Analogy:

Parsing is translating spoken language into intent before acting.

Q3. Walk me through your HTTP parsing logic

Answer structure (this is excellent):

I read the request line first to extract the method, path, and version. Then I parse headers line by line until the CRLF separator. The body is read afterward if present. This mirrors the HTTP/1.1 specification.

Why this sounds senior:

- order matters
- spec awareness
- not guessing

REQUEST → RESPONSE PIPELINE (CORE ARCHITECTURE)

Q4. How does a request become a response in your server?

Ideal answer (verbatim-worthy):

Raw bytes are received from the socket and parsed into an `HttpRequest` object. That request is passed to a routing layer which determines how to handle it. A corresponding `HttpResponse` is generated, serialized back into HTTP wire format, and sent over the socket.

This is **exactly correct**.

Q5. Why separate parsing, routing, and networking?

Strong answer:

Separation of concerns. Parsing understands protocol structure, routing understands application intent, and networking handles I/O. This makes each component simpler, testable, and replaceable.

Hidden signal:

You understand **modular architecture**.

STATIC FILE SERVING (REAL SERVER BEHAVIOR)

Q6. How does your server serve static files?

Strong answer:

The request path is mapped to a file under a configured web root. The server checks an in-memory cache first. If the file is not cached, it is read from disk, cached, and returned as the

response body with the appropriate Content-Type.

Q7. Why do servers cache static files?

Strong answer:

Disk I/O is orders of magnitude slower than memory access. Caching frequently requested files significantly reduces latency and improves throughput.

Extra credit:

- cache avoids syscalls
- improves tail latency

LRU CACHE (DATA STRUCTURES – VERY POPULAR)

Q8. What is an LRU cache?

Strong answer:

An LRU cache evicts the least recently accessed item when capacity is exceeded, under the assumption that recently used data is more likely to be reused.

Q9. How is your LRU cache $O(1)$?

Correct explanation:

It uses a hash map for constant-time lookup and a doubly linked list to maintain access order. On access, items are moved to the front of the list.

If you say this cleanly, interviewers **relax**.

Analogy:

Like keeping recently used tools on top of your desk instead of in a drawer.

Q10. Why not FIFO or LFU?

Answer:

FIFO ignores access patterns, and LFU requires more bookkeeping. LRU offers a good balance of simplicity and real-world effectiveness.

WORKING DIRECTORY BUG (THIS IS A TRICK QUESTION)

Q11. You got a 404 even though the file existed – why?

Excellent answer:

Because the server's working directory differed from the project root. Relative paths were resolved incorrectly. I fixed it by defining an explicit web root relative to the executable's runtime location.

This answer shows **real debugging experience**.

SOCKETS & WINSOCK (SYSTEMS KNOWLEDGE)

Q12. Why does HTTP use TCP?

Strong answer:

TCP provides reliable, ordered, byte-stream delivery, which HTTP relies on to reconstruct complete requests and responses.

Q13. Walk me through the socket lifecycle in your server

Perfect answer:

Initialize Winsock, create a socket, bind it to a port, listen for connections, accept clients, receive data, process requests, send responses, then clean up resources.

This shows **control-flow mastery**.

Q14. Why start single-threaded?

Excellent answer:

Correctness before concurrency. A single-threaded design simplifies reasoning. Once correctness is established, concurrency can be added safely using a thread pool.

This is a **senior mindset**.

THREAD POOLS (PREVIEW — COMING NEXT)

Q15. Why use a thread pool instead of one thread per request?

Strong answer:

A thread-per-request model does not scale and can exhaust system resources. A fixed-size thread pool limits concurrency, improves predictability, and reduces overhead.

MEMORY & C++ DESIGN (VERY IMPORTANT)

Q16. How do you avoid memory leaks?

Strong answer:

I rely on RAII and standard library containers. Resources are acquired and released deterministically via object lifetime.

Q17. Why avoid global variables?

Answer:

Globals introduce hidden coupling and make concurrency unsafe. I use controlled statics only where lifetime must be shared and well-defined.

BIG-PICTURE QUESTIONS (THE ONES THAT DECIDE)

Q18. What did you learn building this?

Excellent answer:

I gained a deep understanding of how HTTP servers work at the byte level, how performance is affected by I/O and caching, and how to structure systems code for correctness before optimization.

Q19. How would you scale this server?

Answer roadmap:

- add thread pool
- non-blocking I/O or IOCP
- connection reuse
- request batching
- better caching strategy

You don't need to implement — just reason.

NOW , We want:

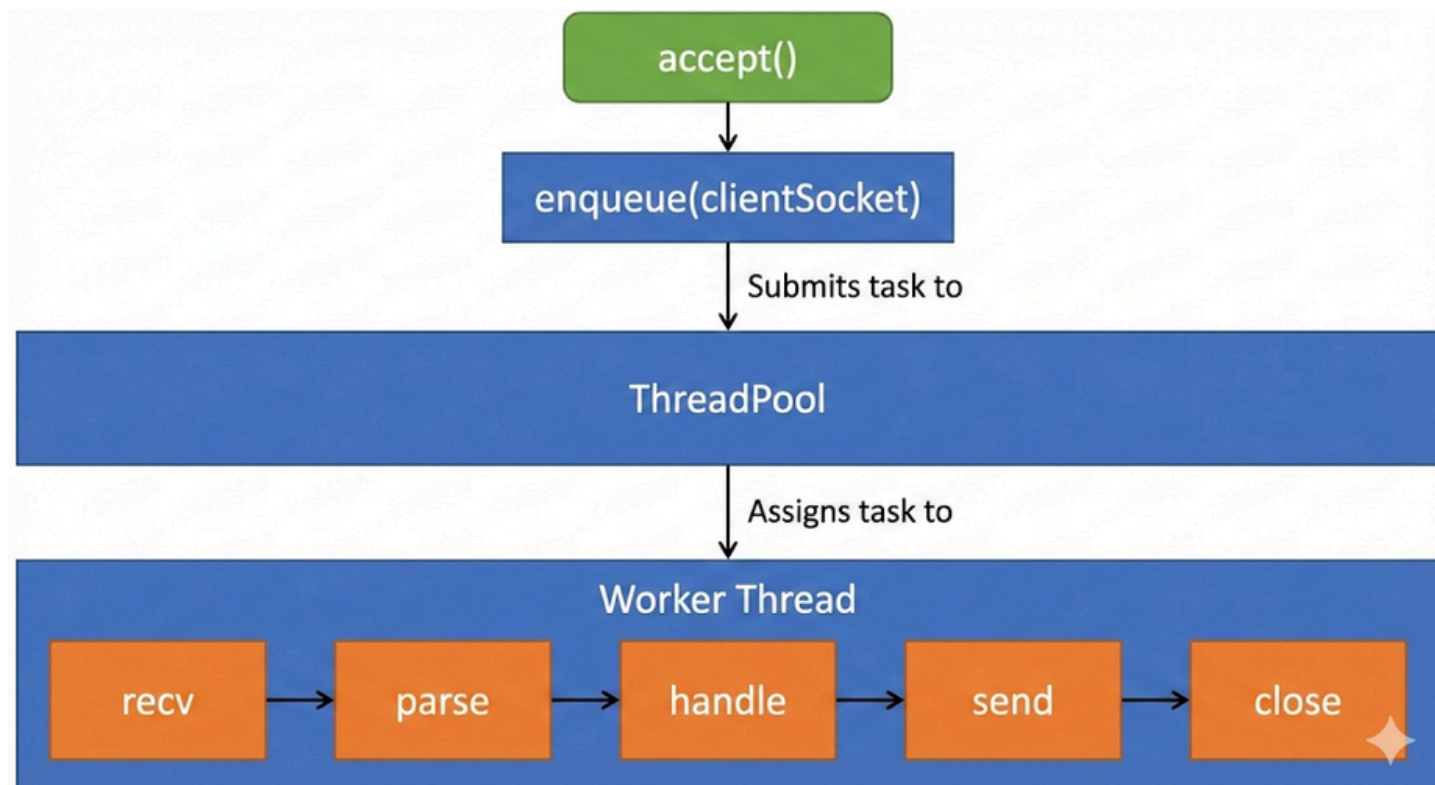
- Multiple clients concurrently
- **Bounded concurrency**
- No thread explosion
- No shared-state races

Rule #1 (important)

👉 The accept loop stays single-threaded

👉 Request handling moves to the thread pool

This is how real servers do it.



Q: Why not accept connections in multiple threads?

Accept is cheap. Handling requests is expensive. Keeping accept single-threaded simplifies design and avoids contention.

Q: What happens if all threads are busy?

New connections queue in the OS accept backlog until a worker becomes available.

Q: Why is this better than async?

Thread pools are simpler to reason about. Async models trade complexity for scalability. This design is a solid baseline.

